NEURONSPARK: A Spiking Neural Network Language Model with Selective State Space Dynamics

Zhengzheng Tang
Boston University
zztangbu@bu.edu

Abstract

We investigate whether large-scale language modeling can be achieved with a fully spiking backbone, without distillation from pretrained Transformers. This question is motivated by the need for architectures that preserve language modeling capacity while offering a clearer path to energy-efficient neuromorphic deployment. We present NEURONSPARK, a 0.9B-parameter spiking language model trained from random initialization via next-token prediction and surrogate-gradient optimization. The architecture combines selective state-space SNN blocks with input-dependent modulation of membrane decay, input gain, and firing threshold, leakage-current activation as the default inter-layer signal, and PonderNet-based adaptive timesteps for token-wise dynamic computation. It further incorporates fused Triton PLIF kernels and stabilization techniques, including residual centering, lateral-inhibition normalization, and natural-gradient compensation. Importantly, inter-layer communication is carried primarily by floating-point leakage-current signals rather than a pure binary spike stream; binary spikes are retained as internal firing events that shape membrane-state evolution. Under a constrained budget (approximately 1.4B pretraining tokens from a 10B-token Chinese corpus and approximately 6.5K SFT steps over 3.5M instruction samples), NEURONSPARK-0.9B reaches a pretraining loss of 3.6 and exhibits early multi-turn dialogue behavior after supervised fine-tuning. These results provide evidence that end-to-end language modeling with a pure SNN architecture is feasible at this scale.

1 Introduction

Large language models (LLMs) based on Transformers [Vaswani et al., 2017] have achieved remarkable success across natural language processing tasks. However, their quadratic attention mechanism and dense floating-point computation raise fundamental questions about computational efficiency and biological plausibility. Meanwhile, spiking neural networks (SNNs) [Maass, 1997] — the "third generation" of neural networks — process information through discrete spikes and temporal dynamics, offering potential advantages in energy efficiency and neuromorphic hardware deployment.

Despite significant progress in SNN-based vision models, SNN language modeling remains underdeveloped. This gap is important because language is a central benchmark for general sequence modeling; without evidence at language-model scale, claims about SNNs as a practical alternative to dense Transformer computation remain limited. Existing approaches such as SpikeGPT [Zhu et al., 2024], SpikingBERT [Bal and Sengupta, 2024], and SpikeBERT [Lv et al., 2023] either rely on distillation from pretrained Transformers, retain non-spiking components in critical stages, or remain at relatively small model scale. Consequently, the field still lacks a clear answer to the following question: *Can a pure SNN architecture learn language from random initialization at meaningful scale under standard next-token training?*

Preprint. Under review.

In this work, we address this gap by introducing NEURONSPARK, a 0.9B-parameter SNN language model trained from random initialization. Given the available compute budget (8× RTX 4090 GPUs), we train on approximately ∼1.4B tokens from a 10B-token corpus; despite this constraint, the model exhibits non-trivial language generation and dialogue behavior. Our key technical insight is that the membrane potential dynamics of Leaky Integrate-and-Fire (LIF) neurons can be formulated as a selective state space model [Gu and Dao, 2024], where the decay rate β, input gain α, and firing threshold V_{th} serve as input-dependent gating mechanisms analogous to Mamba's selection mechanism. This perspective enables us to design an end-to-end spiking language architecture that is both trainable at scale and interpretable through the SSM lens. A key modeling choice is to treat layer-to-layer signals as floating-point leakage-current signals, while retaining 0/1 spikes as the internal neuronal event process; this distinction avoids the expressivity bottleneck of purely binary inter-layer communication.

Contributions.

1. We propose the **Selective State Space SNN Block** with 7 parallel projection paths, computing dynamic $\beta(t), \alpha(t), V_{\text{th}}(t)$ from input signals through learned modulation networks with structured initialization, establishing a formal SNN–SSM duality (Section 3.4).
2. We introduce **leakage-current activation** $(1-\beta) \cdot V_{\text{post}}$ as the default inter-layer signal for PLIFNode boundaries, which naturally emphasizes fast-responding neurons and provides implicit temporal-scale weighting (Section 3.3).
3. We design **PonderNet adaptive timesteps** at each sublayer, enabling per-token dynamic SNN computation depth with geometric-distribution weighting and ponder cost regularization (Section 3.6).
4. We develop **Triton-fused PLIF kernels** with per-element and row-parameter variants, performing the entire PLIF forward/backward (including surrogate gradient) in a single kernel launch (Section 4.3).
5. We introduce **residual centering** and **lateral inhibition normalization** as SNN-native stabilization techniques, along with a two-phase **natural gradient compensation** for modulation parameters (Sections 4.1–4.4).
6. We train and release NEURONSPARK-0.9B under a constrained data budget, and provide evidence that a pure SNN can acquire non-trivial language modeling ability from random initialization (Section 5).

Collectively, these contributions target a single bottleneck in prior work: the lack of a scalable, end-to-end spiking recipe for language modeling that is both theoretically grounded and practically trainable.

2 Related Work

Spiking Neural Networks for Language. Prior work can be grouped by the specific gap it leaves unaddressed. **Distillation dependence**: SpikingBERT [Bal and Sengupta, 2024] and SpikeBERT [Lv et al., 2023] transfer representations from pretrained ANN/Transformer models, which reduces evidence that language competence can emerge from fully spiking training dynamics. **Partial spiking pipelines**: SpikeGPT [Zhu et al., 2024] demonstrates generative behavior with spike-based hidden computation, but still retains non-spiking components (e.g., embedding/output stages), leaving end-to-end spiking feasibility unresolved. **Scale limitations**: existing studies are typically limited to ≤216M parameters, well below contemporary language-model regimes. Our work targets these three gaps jointly by training a 0.9B model from random initialization with standard next-token prediction and spiking dynamics throughout the core sequence-processing stack.

State Space Models. Structured State Spaces (S4) [Gu et al., 2022] introduced efficient linear recurrence for sequence modeling. Mamba [Gu and Dao, 2024] added input-dependent selection, achieving Transformer-competitive performance. Mamba-2 [Dao and Gu, 2024] established a formal duality between SSMs and attention. We observe that SNN membrane dynamics $V[t] = \beta(t) \cdot V[t-1] + \alpha(t) \cdot I[t]$ are structurally identical to the selective SSM recurrence, with β as the decay coefficient and α as the input gate. The spike-and-reset mechanism adds a discrete nonlinearity absent in continuous SSMs.

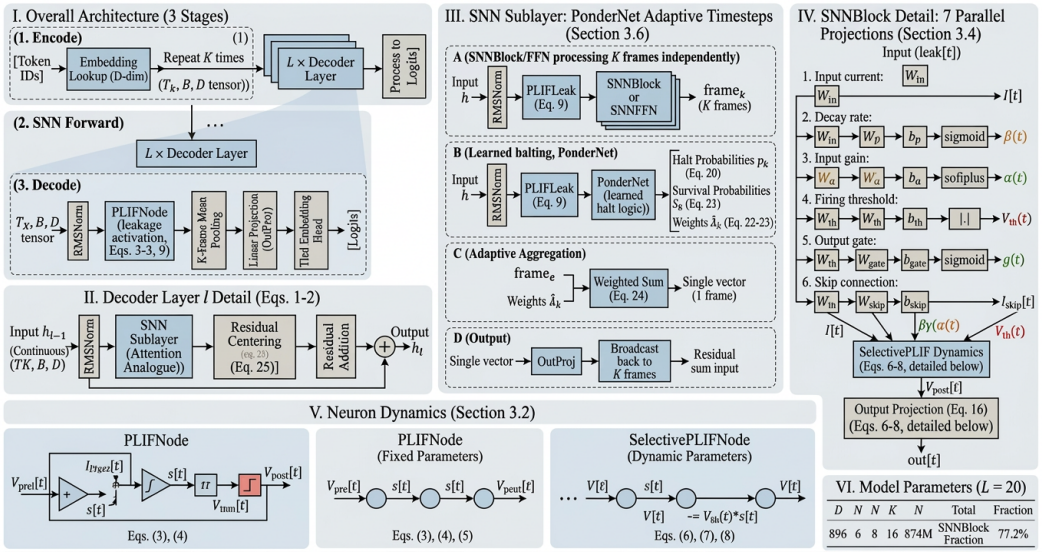

Figure 1: NEURONSPARK architecture overview. The residual stream carries continuous values $\mathbf{h}$; PLIFLeak denotes PLIF neurons with leakage activation $(1-\beta)\cdot V_{\text{post}}$. PonderNet aggregation (applied per sublayer) collapses K frames per token with learned geometric-distribution weights. Inter-layer communication uses floating-point leakage-current signals; binary spikes are internal firing events rather than the default layer-to-layer representation. The decode stage uses uniform K-frame mean. Residual centering (subtract per-token mean) is applied before each residual addition.

Adaptive Computation. Adaptive Computation Time (ACT) [Graves, 2016] allows networks to vary computation per input. PonderNet [Banino et al., 2021] improved upon ACT with a geometric distribution prior. We apply PonderNet at the SNN timestep level within each sublayer: the K frames per token are aggregated with learned halt probabilities, enabling different tokens to use 1 to $K_{\max}$ effective SNN steps.

Surrogate Gradient Training. The non-differentiability of spike generation ($\Theta(V - V_{\text{th}})$) is addressed by surrogate gradient methods [Neftci et al., 2019, Zenke and Vogels, 2021], which replace the Heaviside derivative with smooth approximations. NEURONSPARK uses Sigmoid surrogate gradients ($\alpha = 4.0$) throughout, implemented in the SpikingJelly framework [Fang et al., 2023], with custom Triton kernels that fuse the surrogate computation into the sequential scan.

3 Architecture

This section is organized around one central method question: how to make a pure SNN language model simultaneously expressive, trainable, and scalable. Our design follows a four-step logic: (1) define a stable neuron-level state update, (2) choose an inter-layer signal that avoids binary-communication bottlenecks, (3) build a selective sequence block on top of that signal, and (4) add system-level training stabilizers so optimization remains tractable at 0.9B scale.

3.1 Overview

NEURONSPARK follows a three-stage pipeline (Figure 1): **(1) Encode**: Token IDs $\rightarrow$ embedding (D-dim) $\rightarrow$ repeat K times, producing a $(T{\cdot}K, B, D)$ tensor. Gradients flow directly through embedding; the output head reuses the embedding matrix (weight tying [Press and Wolf, 2017]). **(2) SNN Forward**: L=20 decoder layers with gradient checkpointing, each containing an SNNBlock (attention analogue) and an SNNFFN (MLP analogue), with PonderNet adaptive K-frame aggregation. All neuron states reset per sequence. **(3) Decode**: RMSNorm [Zhang and Sennrich, 2019] $\rightarrow$ output PLIFNode (leakage) $\rightarrow$ K-frame uniform mean $\rightarrow$ projection $\rightarrow$ lateral inhibition [Carandini and Heeger, 2012] $\rightarrow$ tied head $\rightarrow$ logits.

Each decoder layer follows a Pre-LN residual pattern matching Qwen3/LLaMA [Touvron et al., 2023, Yang et al., 2025]:

$$\mathbf{h} \leftarrow \mathbf{h} + \text{center}\big(\text{OutProj}\big(\text{PonderAgg}\big(\text{SNNBlock}\big(\text{PLIFLeak}\big(\text{RMSNorm}(\mathbf{h})\big)\big)\big)\big)\big) \tag{1}$$

$$\mathbf{h} \leftarrow \mathbf{h} + \text{center}\big(\text{OutProj}\big(\text{PonderAgg}\big(\text{SNNFFN}\big(\text{PLIFLeak}\big(\text{RMSNorm}(\mathbf{h})\big)\big)\big)\big) \tag{2}$$

where $\text{center}(\mathbf{x}) = \mathbf{x} - \text{mean}(\mathbf{x})$ is residual centering (Section 4.1). The residual stream $\mathbf{h} \in \mathbb{R}^{TK\times B\times D}$ carries continuous values throughout; only the SNN sublayers operate on spike/membrane dynamics.

The remainder of this section instantiates this logic in order: PLIF dynamics define the base state transition, leakage-current activation defines the default inter-layer representation, SNNBlock/SNNFFN define sequence computation, and the final subsections describe optimization-oriented stabilizers.

3.2 PLIF Neuron Dynamics (Mechanism)

All neurons in NEURONSPARK follow the Parametric Leaky Integrate-and-Fire (PLIF) model [Fang et al., 2021]. This subsection provides the dynamical foundation on which all later architectural choices are built. We distinguish two variants:

PLIFNode (fixed parameters). Used at layer boundaries (input neurons, gate/up neurons, output neuron). Each has D-dimensional (or D_{ff}-dimensional) learnable parameters:

$$V_{\text{pre}}[t] = \beta \cdot V_{\text{post}}[t-1] + (1-\beta)\cdot x[t] \tag{3}$$

$$s[t] = \Theta(V_{\text{pre}}[t] - V_{\text{th}}) \tag{4}$$

$$V_{\text{post}}[t] = V_{\text{pre}}[t] - V_{\text{th}} \cdot s[t] \tag{5}$$

where $\beta = \sigma(w) \in (0,1)$ with $w \sim \mathcal{N}(\text{logit}(1-1/\tau_0),\ 0.5)$ and $V_{\text{th}} \sim \mathcal{U}(0.5v_0,\ 1.5v_0)$. The random initialization creates diversity across dimensions: different neurons have different time constants and firing sensitivities. Equation (5) implements *soft reset*: the membrane potential is reduced by V_{th} upon firing, preserving residual charge.

SelectivePLIFNode (dynamic parameters). Used inside SNNBlock for $D \cdot N$ hidden neurons. Parameters $\beta(t), \alpha(t), V_{\text{th}}(t)$ are computed per-step from the input (Section 3.4):

$$V[t] = \beta(t)\cdot V[t-1] + \alpha(t)\cdot I[t] \tag{6}$$

$$s[t] = \Theta(V[t] - V_{\text{th}}(t)) \tag{7}$$

$$V[t] \mathrel{-}= V_{\text{th}}(t)\cdot s[t] \tag{8}$$

This is structurally identical to Mamba's selective SSM recurrence $h[t] = \bar{A}(t)\cdot h[t-1] + \bar{B}(t)\cdot x[t]$, with the addition of the spike-and-reset nonlinearity.

3.3 Membrane Potential Leakage Activation (Signal)

A critical design choice is the signal transmitted between components. This is the key bridge from neuron dynamics to network-level information flow. Standard SNN practice uses binary spikes $s[t] \in \{0,1\}$, but this severely limits gradient flow through the surrogate function's narrow support. An alternative is the raw membrane potential V_{post}, but this treats all neurons equally regardless of their temporal dynamics.

We use **leakage-current activation** as the default inter-layer signal. In other words, unless explicitly stated otherwise, downstream layers consume floating-point leakage-current signals (bioelectric-state proxies) rather than binary spikes:

$$\text{leak}[t] = (1-\beta)\cdot V_{\text{post}}[t] \tag{9}$$

This quantity is the amount of membrane potential that will *dissipate* due to exponential decay before the next input arrives. Biologically, it corresponds to the leak current through the membrane conductance [Hodgkin and Huxley, 1952, Abbott, 1999].

This leakage-current activation provides natural **temporal-scale weighting**: neurons with large $(1-\beta)$ (fast dynamics, short memory) produce proportionally larger signals, while neurons with small

$(1-\beta)$ (slow dynamics, long memory) are implicitly attenuated. This reweighting is applied at all PLIFNode outputs: input neurons (2 per layer), gate/up neurons in SNNFFN (2 per layer), and the output neuron.

The SelectivePLIFNode *hidden* neurons inside SNNBlock output raw V_{post} rather than leakage, because $\beta(t)$ is dynamic (varies per step) and cannot be absorbed into a static downstream weight matrix. This is a deliberate design choice: leakage scaling applies only at fixed-β boundaries.

3.4 Selective State Space SNN Block (Module)

With neuron dynamics and inter-layer signaling fixed, we next define the core sequence module. The SNNBlock is the attention analogue, processing input through $D \cdot N$ hidden spiking neurons with input-dependent parameters. It computes **seven parallel projections** from the input leakage signal — six input projections and one output projection:

Input projections ($D \to D \cdot N$ or $D \to D$):

$$\mathbf{I}[t] = W_{\text{in}} \cdot \text{leak}[t] \in \mathbb{R}^{DN} \quad \text{(input current)} \tag{10}$$

$$\beta(t) = \sigma(W_\beta \cdot \text{leak}[t] + \mathbf{b}_\beta) \in (0,1)^{DN} \quad \text{(decay rate)} \tag{11}$$

$$\alpha(t) = \text{softplus}(W_\alpha \cdot \text{leak}[t] + \mathbf{b}_\alpha) \in \mathbb{R}_+^{DN} \quad \text{(input gain)} \tag{12}$$

$$V_{\text{th}}(t) = V_{\text{min}} + |W_{\text{th}} \cdot \text{leak}[t] + \mathbf{b}_{\text{th}}| \in [V_{\text{min}}, \infty)^{DN} \quad \text{(threshold)} \tag{13}$$

$$\mathbf{g}[t] = \sigma(W_{\text{gate}} \cdot \text{leak}[t]) \in (0,1)^{D} \quad \text{(output gate)} \tag{14}$$

$$\mathbf{I}_{\text{skip}}[t] = W_{\text{skip}} \cdot \text{leak}[t] \in \mathbb{R}^{D} \quad \text{(skip connection)} \tag{15}$$

The modulation projections $W_\beta, W_\alpha, W_{\text{th}}$ are initialized at $0.1\times$ the scale of W_{in}, ensuring that $\beta(t), \alpha(t), V_{\text{th}}(t)$ are dominated by their respective biases at the start of training, providing a stable initialization.

Hidden neuron dynamics. The $D \cdot N$ hidden neurons follow SelectivePLIF (Eqs. 6–8), computed via fused Triton PLIF kernels (Section 4.3).

Output projection ($D \cdot N \to D$):

$$\text{out}[t] = W_{\text{out}} \cdot V_{\text{post}}[t] \odot \mathbf{g}[t] + \mathbf{I}_{\text{skip}}[t] \tag{16}$$

Note: the output uses V_{post} (not leakage) from the hidden neurons, because $\beta(t)$ is dynamic. The gate $\mathbf{g}$ provides multiplicative control over which dimensions pass through. The skip connection $\mathbf{I}_{\text{skip}}$ ensures gradient flow even when all hidden neurons are silent.

Structured initialization. The modulation biases $\mathbf{b}_\beta, \mathbf{b}_\alpha, \mathbf{b}_{\text{th}}$ receive carefully designed initialization (details in Appendix A): $\mathbf{b}_\beta$ is logit-spaced across N groups targeting $\beta \in [0.80, 0.99]$ (multi-timescale); $\mathbf{b}_\alpha$ is initialized near $\text{softplus}^{-1}(1.0)$ so initial $\alpha \approx 1$; $\mathbf{b}_{\text{th}}$ is calibrated from stationary variance $\sigma_V = \sqrt{p/3} \cdot \sqrt{1-\beta^{2K}}$ with target firing rates 25%–8% across N groups; W_{in} rows are scaled by $\sqrt{1-\beta^2}$ per group; W_{out} columns are scaled by $1/\sqrt{p_{\text{fire}}}$ per group.

3.5 SNN Feed-Forward Network

The SNNFFN mirrors the SwiGLU MLP [Touvron et al., 2023] with spiking neurons replacing the activation function:

$$\text{gate_leak} = (1-\beta_g) \cdot V_{\text{post}}(\text{PLIF}_{\text{gate}}(W_{\text{gate}} \cdot \text{leak})) \tag{17}$$

$$\text{up_leak} = (1-\beta_u) \cdot V_{\text{post}}(\text{PLIF}_{\text{up}}(W_{\text{up}} \cdot \text{leak})) \tag{18}$$

$$\text{out} = W_{\text{down}} \cdot (\text{gate_leak} \odot \text{up_leak}) + W_{\text{skip}} \cdot \text{leak} \tag{19}$$

The element-wise product of two leakage signals replaces $\text{SiLU}(x) \odot x$ gating in SwiGLU [Shazeer, 2020]. Both PLIF neurons provide implicit nonlinearity through the integrate-fire-reset cycle; their leakage outputs carry temporal dynamics that pure activation functions cannot express. W_{down} is initialized with $1/\sqrt{L}$ scaling to prevent gradient explosion through deep residual chains.

3.6 PonderNet Adaptive Timesteps (Module)

Each token is represented as K SNN frames. Rather than uniformly averaging all K frames, we learn per-frame halt probabilities following PonderNet [Banino et al., 2021]:

$$p_k = \sigma(W_{\text{halt}} \cdot \text{frame}_k + b_{\text{halt}}) \in (0, 1) \tag{20}$$

$$S_k = \prod_{j=1}^{k-1}(1 - p_j) \quad \text{(survival probability)} \tag{21}$$

$$\lambda_k = p_k \cdot S_k, \quad \hat{\lambda}_k = \lambda_k / \textstyle\sum_{k'} \lambda_{k'} \tag{22}$$

$$\text{output} = \textstyle\sum_k \hat{\lambda}_k \cdot \text{frame}_k, \quad \mathbb{E}[K] = \textstyle\sum_k k \cdot \hat{\lambda}_k \tag{23}$$

$\mathbb{E}[K]$ serves as a ponder cost regularizer ($\lambda_{\text{ponder}} = 0.01$). PonderNet is applied independently at each sublayer ($2L = 40$ aggregation points). W_{halt} is initialized with Xavier uniform $\times 0.01$ and $b_{\text{halt}} = -3.5$ ($\sigma(-3.5) \approx 0.03$), so PonderNet starts near-uniform and gradually specializes.

After aggregation, the result is projected through OutProj ($D \to D$, no bias), then broadcast back to K frames for residual addition.

4 Stabilization and Efficient Implementation

4.1 Residual Centering (Stabilizer)

Each sublayer's output projection is mean-subtracted before residual addition: $\text{center}(\mathbf{x}) = \mathbf{x} - \frac{1}{D}\sum_{d=1}^{D} x_d$. This eliminates DC drift that would otherwise accumulate across 20 residual layers.

4.2 Lateral Inhibition Normalization (Stabilizer)

The output layer uses **lateral inhibition** (divisive normalization): $\text{LateralInhib}(\mathbf{h}) = \gamma \cdot \mathbf{h}/\sqrt{\frac{1}{D}\sum_d h_d^2 + \epsilon}$, where $\gamma \in \mathbb{R}^D$ is a learnable gain. This is mathematically equivalent to RMSNorm [Zhang and Sennrich, 2019] but corresponds to divisive normalization [Carandini and Heeger, 2012]. We implement it as a fused Triton kernel.

4.3 Triton Fused PLIF Kernels (System)

The PLIF recurrence involves a sequential scan that cannot be trivially parallelized due to spike-and-reset. We implement two variants of fused Triton [Tillet et al., 2019] kernels:

Per-element kernel (SelectivePLIFNode, dynamic $\beta[k], V_{\text{th}}[k]$): single-pass sequential scan with inline charge–fire–reset and Sigmoid surrogate gradient in the backward pass. All arithmetic in fp32 with bf16 storage.

Row-parameter kernel (PLIFNode, fixed β, V_{th}): parameters loaded once into registers, reducing global memory reads from 3 per step to 1, yielding ∼40% speedup. Backward kernel accumulates $\nabla_\beta, \nabla_{V_{\text{th}}}$ in registers.

CPU fallback: 3-phase approach via Hillis-Steele parallel prefix scan [Blelloch, 1990, Martin and Cundy, 2018], spike fixed-point iteration, and surrogate gradient re-computation.

4.4 Natural Gradient Compensation (Stabilizer)

The modulation biases $\mathbf{b}_\beta, \mathbf{b}_\alpha, \mathbf{b}_{\text{th}}$ suffer from two gradient pathologies. We apply compensation after gradient unscaling and before gradient clipping:

Phase 1: Activation saturation. $\nabla_{b_\beta} \leftarrow \nabla_{b_\beta} / \max(\beta(1-\beta),\ 1/C_{\max})$, effectively performing gradient descent in β-space. Similarly for α: $\nabla_{b_\alpha} \leftarrow \nabla_{b_\alpha} / \max(\sigma(b_\alpha), 0.1)$.

Phase 2: Cross-layer equalization. For each modulation parameter type, normalize per-layer gradient norms to the geometric mean: $\nabla_{\text{layer}_i} \leftarrow \nabla_{\text{layer}_i} \cdot \text{GeoMean}(\|\nabla_1\|, \ldots, \|\nabla_L\|)/\|\nabla_i\|$.

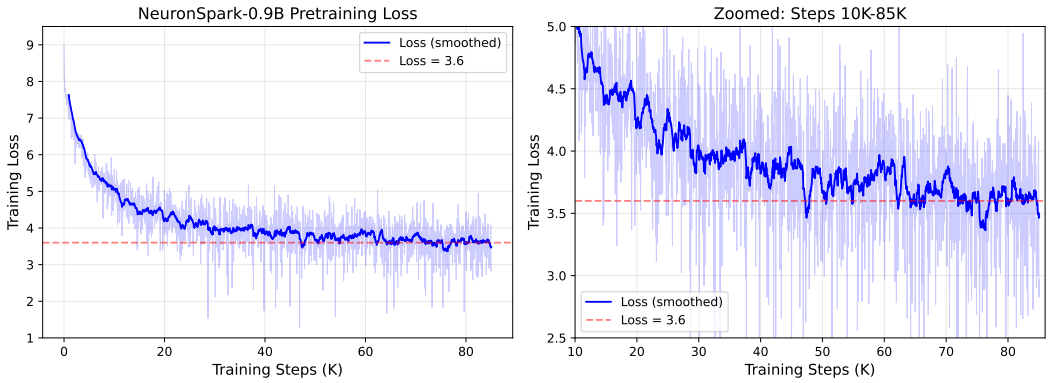

Figure 2: Pretraining loss curve over 85K steps ($\sim$1.4B tokens). Loss decreases from 9.0 to $\sim$3.5. Training throughput: $\sim$960 tokens/sec on 8$\times$ RTX 4090 GPUs.

5 Experiments

5.1 Setup

Model configuration. NeuronSpark-0.9B: $D = 896$, $N = 8$, $K = 16$, $L = 20$, $D_{\text{ff}} = 2688$, 6144-token BPE [Sennrich et al., 2016] vocabulary, 874M parameters.

Datasets. *Pretraining*: Seq-Monkey [Mobvoi, 2023] ($\sim$29M samples, $\sim$10B tokens). *SFT*: BelleGroup train_3.5M_CN [BelleGroup, 2023] ($\sim$3.5M conversations).

Compute constraints. All training was conducted on 8$\times$ NVIDIA RTX 4090 GPUs. Due to limited compute, we train on **small subsets**:

Table 1: Dataset utilization.

Stage	Full Dataset	Actual Used	Fraction
Pretrain	$\sim$10B tokens	85K steps ($\sim$1.4B tokens)	$\sim$14%
SFT	$\sim$3.5M conversations	6.5K steps ($\sim$42K samples)	$\sim$1.2%

Training details. Pretraining: Adam [Loshchilov and Hutter, 2019], peak lr= 2×10^{-4}, 1000-step warmup, cosine decay, gradient accumulation 8, effective batch 64, bfloat16 [Micikevicius et al., 2018], gradient checkpointing [Chen et al., 2016]. Neuron parameters receive $10\times$ base lr. SFT: AdamW (lr= 5×10^{-5}, weight decay 0.01), training only on assistant response tokens.

5.2 Results

Table 2: Training results for NeuronSpark-0.9B.

Metric	Pretrain	SFT
Training loss	3.6	2.1
Parameters	874M	
Training steps	85,000	6,500
Tokens seen	$\sim$1.4B	$\sim$0.4B
Hardware	8$\times$ NVIDIA RTX 4090	

Qualitative evaluation. After SFT, the model demonstrates basic Chinese dialogue (translated; model outputs in Chinese):

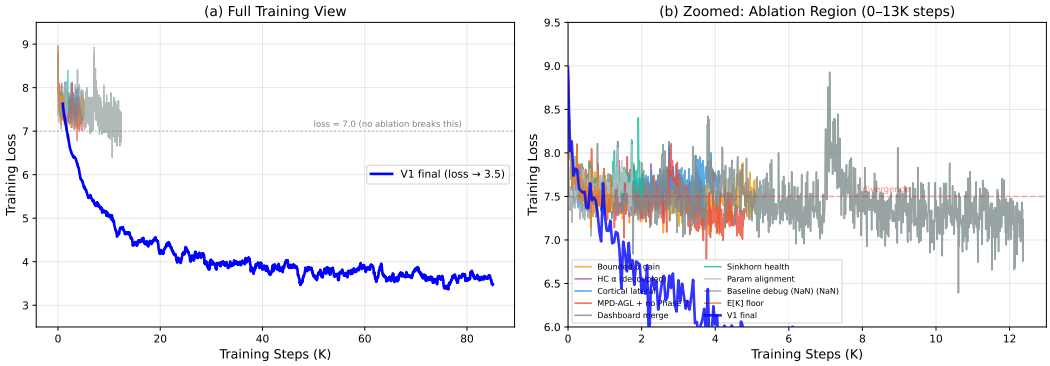

Figure 3: Training loss: final architecture (blue) vs. 9 ablation variants. None of the ablation variants achieves a loss below 7.0.

Q: What is the capital of China? **A:** The capital of China is Beijing.
Q: Hello! **A:** How can I help you?

These observations suggest that a pure SNN architecture can support coherent language generation from random initialization, even under limited-data training.

5.3 Architecture Ablation via Training Stability

During development, we explored multiple architectural variants (each trained 1K–12K steps). Table 3 summarizes 7 variants; Figure 3 shows loss curves.

Table 3: Ablation variants. All stagnated above loss 7.0; only the final architecture reached 3.5.

Variant	Steps	Loss	What Changed
Final V1	**85K**	**3.5**	**Full architecture**
MPD-AGL + no Phase 2	4.8K	7.21	Adaptive surrogate gradient, removed cross-layer equalization
E[K] floor	1.2K	7.47	Added minimum E[K] floor
Bounded α	5.1K	7.47	Bounded gain multiplier
HC α (decoupled)	3.7K	7.44	Separate α parameter
Sinkhorn health	2.1K	7.62†	Sinkhorn-projected health score
Cortical lateral	4.1K	7.66†	Cross-token causal spike propagation
No gradient sync	0.6K	NaN	Missing gradient synchronization

†Diverged (loss $>$7.5).

5.4 Comparison with Existing SNN Language Models

Table 4: Comparison with existing SNN language models.

Model	Params	From Scratch	SNN Core	Generative	Dialogue
SpikeBERT	110M	✗	✓	✗	✗
SpikingBERT	110M	✗	✓	✗	✗
SpikeGPT	216M	✓	✓	✓	✗
NeuronSpark-0.9B	**874M**	✓	✓	✓	✓

6 Discussion

Our central claim is that large-scale language modeling in a pure SNN regime is not only conceptually plausible but empirically attainable when the architectural design directly addresses the optimization and expressivity gaps left by prior SNN language studies.

SNN–SSM duality. The selective PLIF dynamics establish a direct correspondence with Mamba's selective SSM: $\beta(t) \leftrightarrow \bar{A}(t)$, $\alpha(t) \leftrightarrow \bar{B}(t)$, $V[t] \leftrightarrow h[t]$. The spike-and-reset mechanism introduces a hard, input-dependent nonlinearity absent in continuous SSMs.

Data efficiency. NEURONSPARK acquires basic language capabilities with $\sim$14% of pretraining data and $\sim$1.2% of SFT data, suggesting favorable inductive biases, though controlled Transformer baselines are needed.

Limitations. (1) 0.9B parameters, 512-token context. (2) No quantitative benchmarks (C-Eval, CMMLU). (3) Chinese only. (4) Repetition artifacts.

Energy efficiency. The spike-based hidden computation may be amenable to deployment on neuromorphic platforms (e.g., Intel Loihi [Davies et al., 2018]), which could yield substantial energy savings. A rigorous quantitative evaluation remains future work.

7 Conclusion

We presented NEURONSPARK, a 0.9B-parameter spiking language model that jointly addresses three persistent gaps in prior work: distillation dependence, partially non-spiking pipelines, and limited model scale. By connecting SNN membrane dynamics to selective state space models and introducing leakage-current inter-layer signaling, PonderNet adaptive timesteps, fused Triton PLIF kernels, residual centering, lateral inhibition normalization, and natural-gradient compensation, we show that pure SNN architectures can learn non-trivial language behavior from random initialization under limited training data.

Code, weights, and training infrastructure are publicly available[1].

References

Larry F Abbott. Lapicque's introduction of the integrate-and-fire model neuron (1907). *Brain Research Bulletin*, 50(5-6):303–304, 1999.

Malyaban Bal and Abhronil Sengupta. Spikingbert: Distilling bert to train spiking language models using implicit differentiation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38 (10):10998–11006, 2024.

Andrea Banino, Jan Balaguer, and Charles Blundell. Pondernet: Learning to ponder. *International Conference on Machine Learning Workshop on Theoretic Foundation, Criticism, and Application Trend of Explainable AI*, 2021.

BelleGroup. Bellegroup train_3.5m_cn: Chinese instruction-following dataset, 2023. URL `https://huggingface.co/datasets/BelleGroup/train_3.5M_CN`.

Guy E Blelloch. Prefix sums and their applications. *Technical Report CMU-CS-90-190, School of Computer Science, Carnegie Mellon University*, 1990.

Matteo Carandini and David J Heeger. Normalization as a canonical neural computation. *Nature Reviews Neuroscience*, 13(1):51–62, 2012.

Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *International Conference on Machine Learning*, 2024.

Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018.

[1]Code: `https://github.com/Brain2nd/NeuronSpark`; HuggingFace: `https://huggingface.co/Brain2nd`; ModelScope: `https://modelscope.ai/organization/Brain2nd`.

Wei Fang, Zhaofei Yu, Yanqi Chen, Timothee Masquelier, Tiejun Huang, and Yonghong Tian. Incorporating learnable membrane time constant to enhance learning of spiking neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2661–2671, 2021.

Wei Fang, Yanqi Chen, Jianhao Ding, Zhaofei Yu, Timothée Masquelier, Ding Chen, Liwei Huang, Huihui Zhou, Guoqi Li, and Yonghong Tian. Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence. *Science Advances*, 9(40):eadi1480, 2023.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *International Conference on Machine Learning*, 2024.

Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *International Conference on Learning Representations*, 2022.

Alan L Hodgkin and Andrew F Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of Physiology*, 117(4):500–544, 1952.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations*, 2019.

Changze Lv, Tianlong Xu, Jianhan Li, Chenxi Wang, and Jian Liu. Spikebert: A language spikformer trained with two-stage knowledge distillation from bert. *arXiv preprint arXiv:2308.15122*, 2023.

Wolfgang Maass. Networks of spiking neurons: the third generation of neural network models. *Neural Networks*, 10(9):1659–1671, 1997.

Eric Martin and Chris Cundy. Parallelizing linear recurrent neural nets over sequence length. *International Conference on Learning Representations*, 2018.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. *International Conference on Learning Representations*, 2018.

Mobvoi. Seq-monkey general open corpus, 2023. URL `https://modelscope.cn/datasets/ddzhu123/seq-monkey`.

Emre O Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019.

Ofir Press and Lior Wolf. Using the output embedding to improve language models. *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, pages 157–163, 2017.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 2019.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725, 2016.

Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. *Proceedings of the 3rd MLSys Conference*, 2019.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

An Yang, Baosong Yang, Beichen Zhang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Friedemann Zenke and Tim P Vogels. The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks. *Neural Computation*, 33(4):899–925, 2021.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

Rui-Jie Zhu, Qihang Zhao, Guoqi Li, and Jason K Eshraghian. Spikegpt: Generative pre-trained language model with spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 2024.

A Structured Initialization Details

The SNNBlock modulation parameters require careful initialization. We target $K_{\text{ref}} = 16$ steps and assumed input firing rate $p = 0.15$.

Multi-timescale β: $\beta_n = \text{linspace}(0.80, 0.99, N)$; bias $b_{\beta,\,n} = \log(\beta_n/(1-\beta_n))$, repeated across D channels with $\mathcal{N}(0, 0.1)$ perturbation.

α **near unity**: $b_\alpha \sim \mathcal{N}(0.5413, 0.1)$, giving $\alpha \approx 1.0$.

Threshold calibration: $\sigma_V(\beta) = \sqrt{p/3} \cdot \sqrt{1-\beta^{2K_{\text{ref}}}}$; target firing rates $p_{\text{fire}} = \text{linspace}(0.25, 0.08, N)$; $V_{\text{th},n} = \sigma_V(\beta_n) \cdot \Phi^{-1}(1 - p_{\text{fire},n})$.

$W_{\textbf{in}}$ **scaling**: rows scaled by $\sqrt{1-\beta_n^2}$ per group. $W_{\textbf{out}}$ **balancing**: columns scaled by $1/\sqrt{p_{\text{fire},n}}$ (normalized to mean 1).

B Model Configuration

Table 5: Detailed model configuration.

Hidden dim (D)	896
State expansion (N)	8
Max SNN steps (K)	16
Layers (L)	20
FFN dim (D_{ff})	2688
Vocab	6144
Context	512
Total params	874M
Surrogate	Sigmoid(α=4.0)
V_{min}	0.1
Neuron LR mult	10×
Ponder weight	0.01
Output proj init	$0.02/\sqrt{2L}$ (GPT-2 [Radford et al., 2019])

C Parameter Breakdown

Table 6: Parameter breakdown. SNNBlock dominates (77.2%) due to 7 projections in $D \times N$ space.

Component	Params	%
Embedding (tied)	5.5M	0.6
SNNBlock $\times 20$	674.8M	77.2
SNNFFN $\times 20$	160.8M	18.4
Residual proj $\times 40$	32.1M	3.7
Other	1.0M	0.1
Total	**874.1M**	**100**

D Engineering Optimizations

- **Fused modulation**: σ, softplus, $|\cdot|$, $\times$ fused via `torch.compile` into single kernel.
- **Fused halt weights**: PonderNet $\sigma \to \log(1-p) \to$ cumsum $\to \exp \to$ normalize fused.
- **Merged SNNFFN matmul**: $W_{\text{gate}}, W_{\text{up}}$ concatenated into single $(2D_{\text{ff}}, D)$ matmul.
- **Merged PLIF scan**: Gate/up neurons merged into single $2D_{\text{ff}}$-dim scan.
- **Gradient checkpointing**: Each of L=20 layers checkpointed ($\sim$60% memory reduction).

E Training Hyperparameters

Table 7: Training hyperparameters.

	Pretrain	SFT
Optimizer	Adam	AdamW
Peak LR	2×10^{-4}	5×10^{-5}
Neuron LR	2×10^{-3}	5×10^{-4}
Schedule	Warmup+Cosine	Warmup+Cosine
Warmup	1000	100
Weight decay	0	0.01
Eff. batch	64	64
Grad clip	1.0	1.0
Precision	bf16	bf16